

SISTEM INFORMASI RUMAH SAKIT



DESKRIPSI SISTEM

Rumah sakit merupakan institusi yang menangani berbagai aktivitas pelayanan kesehatan, mulai dari pendaftaran pasien, pemeriksaan dokter, pencatatan rekam medis, pengelolaan obat, rawat inap, hingga proses pembayaran. Banyaknya data dan transaksi yang terjadi setiap hari membutuhkan sistem yang mampu mengelola informasi secara terintegrasi. Oleh karena itu, diperlukan Sistem Informasi Rumah Sakit yang dapat membantu proses pengelolaan data menjadi lebih cepat, akurat, dan efisien serta mendukung peningkatan kualitas pelayanan kepada pasien.

TUJUAN PROJECT

- Merancang basis data untuk Sistem Informasi Rumah Sakit yang terintegrasi.
- Mempermudah pengelolaan data pasien, dokter, poliklinik, dan kamar.
- Mendukung pencatatan kunjungan, rekam medis, dan resep obat secara terstruktur.
- Mengelola proses rawat inap, tagihan, dan pembayaran secara efektif.
- Menyediakan informasi yang akurat untuk mendukung operasional rumah sakit.

RUANG LINGKUP SISTEM

- Merancang basis data untuk Sistem Informasi Rumah Sakit yang terintegrasi.
- Mempermudah pengelolaan data pasien, dokter, poliklinik, dan kamar.
- Mendukung pencatatan kunjungan, rekam medis, dan resep obat secara terstruktur.
- Mengelola proses rawat inap, tagihan, dan pembayaran secara efektif.
- Menyediakan informasi yang akurat untuk mendukung operasional rumah sakit.

BUSINESS RULES

PASIEN

- Setiap pasien punya ID & No. RM unik
- Dapat daftar lebih dari satu kali
- Riwayat tetap tersimpan meski tidak aktif

DOKTER DAN PERAWAT

- SIP dokter & STR perawat harus unik
- Dokter nonaktif tidak bisa terima pasien
- Perawat memiliki shift:
pagi/siang/malam

OBAT DAN RESEP

- Setiap obat punya stok_minimum
- Resep wajib terhubung ke rekam medis
- Trigger otomatis kurangi stok saat insert resep

TAGIHAN

- Total = biaya dokter + obat + tindakan + kamar
- Metode: tunai / kartu / BPJS / asuransi
- Status: belum_lunas / lunas

RAWAT INAP

- Terhubung ke dokter, perawat, ruang rawat
- Lama inap = tanggal keluar - tanggal masuk
- Kondisi pasien saat keluar wajib dicatat

INTEGRITAS DATA

- PK untuk semua tabel utama
- FK menjaga referential integrity
- Seluruh alur dapat ditelusuri dari daftar
→ tagihan

ENTITAS & ATRIBUT UTAMA

PASIEN

id_pasien (PK), no_rm, nama_lengkap,
tanggal_lahir, jenis_kelamin, alamat,
no_telepon

POLI

id_poli (PK), nama_poli, lantai, kepala_poli
(FK→Dokter)

PERAWAT

id_perawat (PK), nama_perawat, no_str
(Unique), divisi, shift, status

DOKTER

id_dokter (PK), nama_dokter, spesialisasi,
no_sip (Unique), status

RUANG_PERAWAT

id_ruang (PK), nama_ruang, kelas (VIP/I/II/III),
kapasitas, harga_per_malam

PENDAFTARAN

id_daftar (PK), id_pasien (FK), id_poli (FK),
id_dokter (FK), no_antrian, status

REKAM_MEDIS

id_rekam (PK), id_daftar (FK), id_pasien (FK),
id_dokter (FK), diagnosa, tindakan

OBAT

id_obat (PK), nama_obat, kategori, satuan,
harga, stok, stok_minimum

RESEP

id_resep (PK), id_rekam (FK), id_obat (FK),
jumlah, aturan_pakai

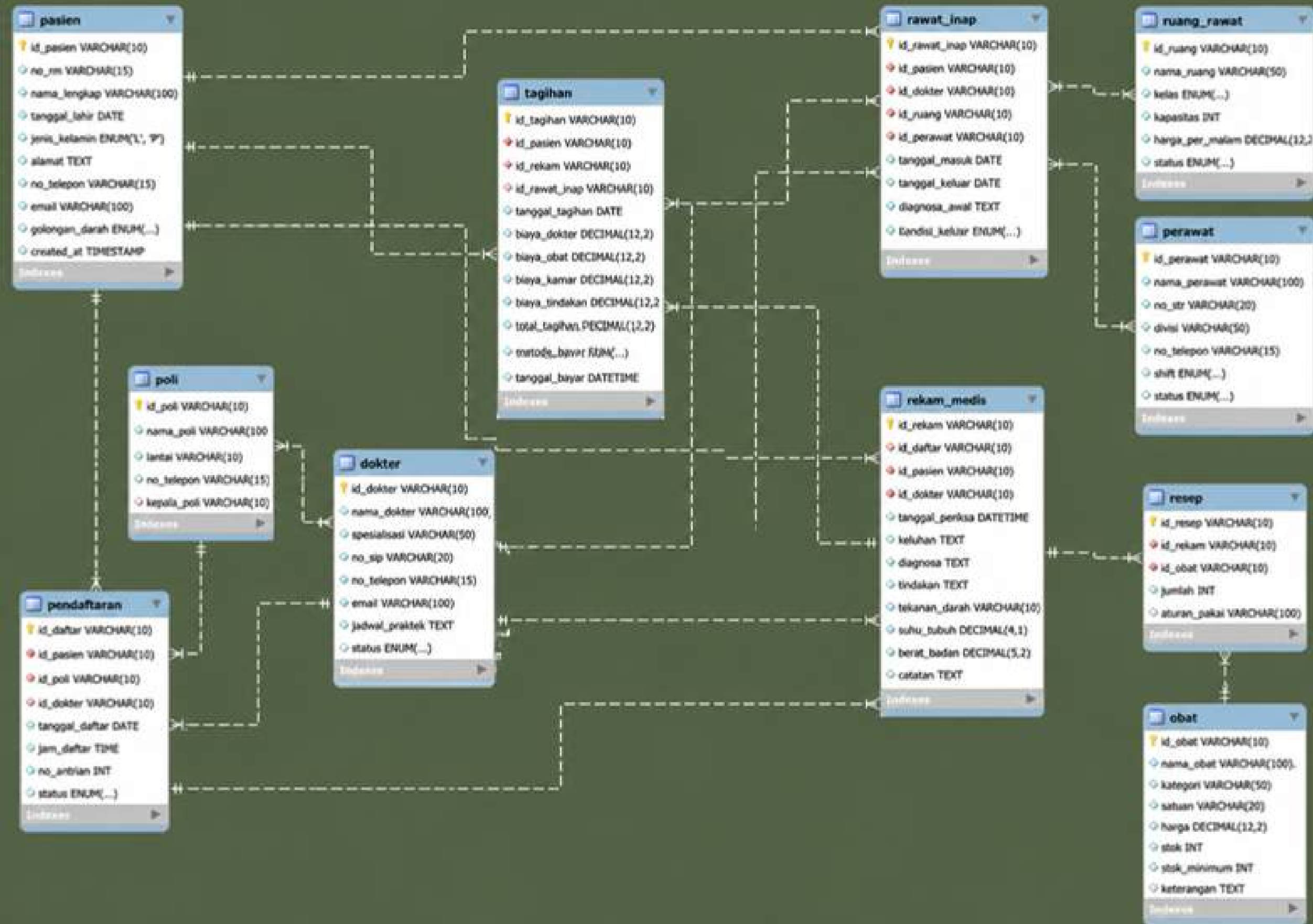
RAWAT INAP

id_rawat_inap (PK), id_pasien (FK), id_dokter
(FK), id_ruang (FK), id_perawat (FK)

TAGIHAN

id_tagihan (PK), id_pasien (FK), total_tagihan,
metode_bayar, status_bayar

ENTITY RELATIONSHIP DIAGRAM



PROSES NORMALISASI

UNF

UNF — Masalah

- ▶ Satu kolom berisi banyak nilai (multivalued)
- ▶ Redundansi data dokter, poli, dan ruang
- ▶ Sulit update, query, dan rawan inkonsistensi

1NF

1NF — Solusi

- ▶ Tiap kolom hanya menyimpan satu nilai
- ▶ Tidak ada atribut multivalued
- ▶ Data obat yang berganda dipisah ke baris baru

2NF

2NF — Solusi

- ▶ Tidak ada partial dependency
- ▶ Atribut non-key bergantung penuh pada PK
- ▶ Tabel dipecah: PASIEN, DOKTER, POLI, dst.

3NF

3NF — Hasil

- ▶ Tidak ada transitive dependency
- ▶ Semua atribut non-key → PK langsung
- ▶ 11 tabel final: PASIEN, DOKTER, POLI, PERAWAT, RUANG_RAWAT, PENDAFTARAN, REKAM_MEDIS, OBAT, RESEP, RAWAT_INAP, TAGIHAN

DATA DICTIONARY & DDL

Struktur tabel utama berdasarkan hasil normalisasi 3NF

```
-- 1. PASIEN
CREATE TABLE pasien (
  id_pasien    VARCHAR(10) PRIMARY KEY,
  nama_lengkap VARCHAR(100) NOT NULL,
  tanggal_lahir DATE NOT NULL,
  jenis_kelamin ENUM('L', 'P') NOT NULL,
  alamat       TEXT,
  no_telepon    VARCHAR(15),
  email         VARCHAR(100),
  golongan_darah ENUM('A', 'B', 'AB', 'O'),
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. DOKTER
CREATE TABLE dokter (
  id_dokter    VARCHAR(10) PRIMARY KEY,
  nama_dokter  VARCHAR(100) NOT NULL,
  spesialisasi VARCHAR(50) NOT NULL,
  no_sip       VARCHAR(20) UNIQUE NOT NULL,
  no_telepon   VARCHAR(15),
  email        VARCHAR(100),
  jadwal_praktek TEXT,
  status       ENUM('aktif', 'nonaktif') DEFAULT 'aktif'
);

-- 3. PERAWAT
CREATE TABLE perawat (
  id_perawat   VARCHAR(10) PRIMARY KEY,
  nama_perawat VARCHAR(100) NOT NULL,
  no_str       VARCHAR(20) UNIQUE NOT NULL,
  divisi       VARCHAR(50),
  no_telepon   VARCHAR(15),
  shift        ENUM('pagi', 'siang', 'malam'),
  status       ENUM('aktif', 'nonaktif') DEFAULT 'aktif'
);
```

```
-- 4. POLI / DEPARTEMEN
CREATE TABLE poli (
  id_poli      VARCHAR(10) PRIMARY KEY,
  nama_poli    VARCHAR(100) NOT NULL,
  lantai       VARCHAR(10),
  no_telepon   VARCHAR(15),
  kepala_poli  VARCHAR(10),
  FOREIGN KEY (kepala_poli) REFERENCES dokter(id_dokter)
);

-- 5. RUANG RAWAT INAP
CREATE TABLE ruang_rawat (
  id_ruang     VARCHAR(10) PRIMARY KEY,
  nama_ruang   VARCHAR(50) NOT NULL,
  kelas        ENUM('VIP', 'I', 'II', 'III') NOT NULL,
  kapasitas    INT NOT NULL,
  harga_per_malam DECIMAL(12,2) NOT NULL,
  status       ENUM('tersedia', 'penuh') DEFAULT 'tersedia'
);
```

```
-- 6. PENDAFTARAN / REGISTRASI
CREATE TABLE pendaftaran (
  id_daftar    VARCHAR(10) PRIMARY KEY,
  id_pasien    VARCHAR(10) NOT NULL,
  id_poli      VARCHAR(10) NOT NULL,
  id_dokter    VARCHAR(10) NOT NULL,
  tanggal_daftar DATE NOT NULL,
  jam_daftar   TIME NOT NULL,
  no_antrian   INT NOT NULL,
  status       ENUM('menunggu', 'dilayani', 'selesai', 'batal') DEFAULT 'menunggu',
  FOREIGN KEY (id_pasien) REFERENCES pasien(id_pasien),
  FOREIGN KEY (id_poli) REFERENCES poli(id_poli),
  FOREIGN KEY (id_dokter) REFERENCES dokter(id_dokter)
);
```

```
-- 7. REKAM MEDIS
CREATE TABLE rekam_medis (
  id_rekam     VARCHAR(10) PRIMARY KEY,
  id_daftar    VARCHAR(10) NOT NULL,
  id_pasien    VARCHAR(10) NOT NULL,
  id_dokter    VARCHAR(10) NOT NULL,
  tanggal_periksa DATETIME NOT NULL,
  keluhan      TEXT,
  diagnosa     TEXT NOT NULL,
  tindakan     TEXT,
  tekanan_darah VARCHAR(15),
  suhu_tubuh   DECIMAL(4,1),
  berat_badan  DECIMAL(5,2),
  catatan      TEXT,
  FOREIGN KEY (id_daftar) REFERENCES pendaftaran(id_daftar),
  FOREIGN KEY (id_pasien) REFERENCES pasien(id_pasien),
  FOREIGN KEY (id_dokter) REFERENCES dokter(id_dokter)
);
```

```
-- 8. RAWAT INAP
CREATE TABLE rawat_inap (
  id_rawat_inap VARCHAR(10) PRIMARY KEY,
  id_pasien     VARCHAR(10) NOT NULL,
  id_dokter     VARCHAR(10) NOT NULL,
  id_ruang      VARCHAR(10) NOT NULL,
  id_perawat    VARCHAR(10) NOT NULL,
  tanggal_masuk DATE NOT NULL,
  tanggal_keluar DATE,
  diagnosa_awal TEXT,
  diagnosa_akhir TEXT,
  kondisi_keluar ENUM('sembuh', 'dirujuk', 'meninggal', 'atas_permintaan'),
  FOREIGN KEY (id_pasien) REFERENCES pasien(id_pasien),
  FOREIGN KEY (id_dokter) REFERENCES dokter(id_dokter),
  FOREIGN KEY (id_ruang) REFERENCES ruang_rawat(id_ruang),
  FOREIGN KEY (id_perawat) REFERENCES perawat(id_perawat)
);
```

```
-- 9. OBAT & FARMASI
CREATE TABLE obat (
  id_obat      VARCHAR(10) PRIMARY KEY,
  nama_obat    VARCHAR(100) NOT NULL,
  kategori     VARCHAR(50),
  satuan       VARCHAR(20) NOT NULL,
  harga        DECIMAL(12,2) NOT NULL,
  stok         INT NOT NULL DEFAULT 0,
  stok_minimum INT DEFAULT 10,
  keterangan   TEXT
);

-- RESEP (relasi rekam medis dan obat)
CREATE TABLE resep (
  id_resep     VARCHAR(10) PRIMARY KEY,
  id_rekam     VARCHAR(10) NOT NULL,
  id_obat      VARCHAR(10) NOT NULL,
  jumlah       INT NOT NULL,
  aturan_pakai VARCHAR(100),
  FOREIGN KEY (id_rekam) REFERENCES rekam_medis(id_rekam),
  FOREIGN KEY (id_obat) REFERENCES obat(id_obat)
);
```

```
-- 10. TAGIHAN / PEMBAYARAN
CREATE TABLE tagihan (
  id_tagihan   VARCHAR(10) PRIMARY KEY,
  id_pasien    VARCHAR(10) NOT NULL,
  id_rekam     VARCHAR(10),
  id_rawat_inap VARCHAR(10),
  tanggal_tagihan DATE NOT NULL,
  biaya_dokter DECIMAL(12,2) DEFAULT 0,
  biaya_obat   DECIMAL(12,2) DEFAULT 0,
  biaya_kamar  DECIMAL(12,2) DEFAULT 0,
  biaya_tindakan DECIMAL(12,2) DEFAULT 0,
  total_tagihan DECIMAL(12,2) NOT NULL,
  metode_bayar ENUM('tunai', 'kartu', 'bpjs', 'asuransi'),
  status_bayar  ENUM('belum_lunas', 'lunas') DEFAULT 'belum_lunas',
  tanggal_bayar DATETIME,
  FOREIGN KEY (id_pasien) REFERENCES pasien(id_pasien),
  FOREIGN KEY (id_rekam) REFERENCES rekam_medis(id_rekam),
  FOREIGN KEY (id_rawat_inap) REFERENCES rawat_inap(id_rawat_inap)
);
```

PRIMARY KEY

ID unik tiap entitas (tidak boleh NULL, tidak boleh duplikat)

FOREIGN KEY

Menjaga referential integrity antar tabel

UNIQUE

No. RM, No. SIP, No. STR tidak boleh sama

NOT NULL

Data wajib seperti nama, tanggal lahir, diagnosa

ENUM

Membatasi nilai: jenis kelamin, status, shift, kelas

IMPLEMENTASI & DDM

Implementasi data ke dalam tabel hasil normalisasi

```
1  -- =====
2  -- INSERT DOKTER (10 data)
3  -- =====
4  • INSERT INTO dokter VALUES
5  ('D001', 'dr. Hendra Kusuma, Sp.PD', 'Penyakit Dalam', 'SIP/2015/001/JTG', '081234560001', 'he
6  ('D002', 'dr. Siti Rahayu, Sp.A', 'Anak', 'SIP/2016/002/JTG', '081234560002', 's
7  ('D003', 'dr. Bambang Wibowo, Sp.OG', 'Obstetri & Ginekologi', 'SIP/2014/003/JTG', '081234560003', 'b
8  ('D004', 'dr. Dewi Permatasari, Sp.JP', 'Jantung & Pembuluh Darah', 'SIP/2017/004/JTG', '081234560004', 'd
9  ('D005', 'dr. Agus Santoso, Sp.B', 'Bedah Umum', 'SIP/2013/005/JTG', '081234560005', 'a
10 ('D006', 'dr. Ratna Dewi, Sp.S', 'Saraf', 'SIP/2018/006/JTG', '081234560006', 'r
11 ('D007', 'dr. Fajar Nugroho, Sp.M', 'Mata', 'SIP/2019/007/JTG', '081234560007', 'f
12 ('D008', 'dr. Lina Marlina, Sp.KK', 'Kulit & Kelamin', 'SIP/2020/008/JTG', '081234560008', 'l
13 ('D009', 'dr. Rudi Hartono, Sp.THT', 'THT', 'SIP/2016/009/JTG', '081234560009', 'r
14 ('D010', 'dr. Yunita Sari, Sp.Rad', 'Radiologi', 'SIP/2017/010/JTG', '081234560010', 'y
15
16  -- =====
17  -- INSERT POLI (8 data)
18  -- =====
19  • INSERT INTO poli VALUES
20  ('P001', 'Poli Penyakit Dalam', 'Lantai 1', '024-8410001', 'D001'),
21  ('P002', 'Poli Anak', 'Lantai 1', '024-8410002', 'D002'),
22  ('P003', 'Poli Kandungan', 'Lantai 2', '024-8410003', 'D003'),
23  ('P004', 'Poli Jantung', 'Lantai 2', '024-8410004', 'D004'),
24  ('P005', 'Poli Bedah', 'Lantai 3', '024-8410005', 'D005'),
25  ('P006', 'Poli Saraf', 'Lantai 2', '024-8410006', 'D006'),
26  ('P007', 'Poli Mata', 'Lantai 1', '024-8410007', 'D007'),
```

INSERT DATA MASTER

Pasien, Dokter, Perawat, Poli, Ruang, Obat

INSERT PENDAFTARAN

Pasien daftar ke poli & dokter tujuan

INSERT REKAM MEDIS

Dokter catat keluhan, diagnosa, tindakan

INSERT RESAP

Trigger otomatis kurangi stok obat

INSERT RAWAT INAP

Jika perlu: dokter, perawat, ruang

INSERT TAGIHAN

Total biaya dokter + obat + kamar

CONTOH HASIL OUTPUT UNTUK DATA PASIEN

	id_pasien	nama_lengkap	tanggal_lahir	jenis_kelamin	alamat	no_telepon	email	golongan_darah	created_at
▶	PS001	Budi Santoso	1978-04-12	L	Jl. Merdeka No. 14, Semarang	081311110001	budi.santoso@gmail.com	A	2024-01-05 08:30:00
	PS002	Siti Aminah	1985-07-23	P	Jl. Pahlawan No. 7, Semarang	081311110002	siti.aminah@gmail.com	O	2024-01-08 09:15:00
	PS003	Ahmad Fauzi	1992-11-03	L	Jl. Sudirman No. 22, Ungaran	081311110003	ahmad.fauzi@yahoo.com	B	2024-01-10 10:00:00
	PS004	Dwi Lestari	1990-02-17	P	Jl. Diponegoro No. 45, Kendal	081311110004	dwi.lestari@gmail.com	AB	2024-01-12 08:45:00
	PS005	Heru Prasetyo	1975-09-08	L	Jl. Gajah Mada No. 3, Demak	081311110005	heru.pras@gmail.com	A	2024-01-15 11:00:00
	PS006	Nur Hidayah	1995-06-30	P	Jl. Ahmad Yani No. 18, Semarang	081311110006	nur.hidayah@gmail.com	O	2024-01-18 09:30:00
	PS007	Wahyu Triadi	1988-03-25	L	Jl. Pemuda No. 55, Semarang	081311110007	wahyu.triadi@gmail.com	B	2024-01-20 10:30:00
	PS008	Eka Putri Rahayu	2001-12-14	P	Jl. Veteran No. 9, Salatiga	081311110008	eka.putri@gmail.com	A	2024-01-22 08:00:00
	PS009	Joko Susilo	1965-08-19	L	Jl. Wolter Monginsidi No. 33, Semarang	081311110009	NULL	O	2024-01-25 13:00:00
	PS010	Ani Suryani	1980-05-07	P	Jl. Brigjen Katamso No. 12, Semarang	081311110010	ani.suryani@gmail.com	AB	2024-02-01 08:30:00
	PS011	Rizki Maulana	2000-01-20	L	Jl. Cempaka No. 6, Ungaran	081311110011	rizki.maulana@gmail.com	A	2024-02-03 09:00:00
	PS012	Fitriani Dewi	1993-10-11	P	Jl. Teuku Umar No. 27, Semarang	081311110012	fitriani.dewi@gmail.com	B	2024-02-05 10:15:00
	PS013	Supardi	1958-06-04	L	Jl. Raya Kaligawe No. 88, Semarang	081311110013	NULL	O	2024-02-07 08:00:00
	PS014	Yanti Kusumawati	1987-04-28	P	Jl. Singosari No. 14, Semarang	081311110014	yanti.kusumaw@gmail.com	A	2024-02-10 11:30:00
	PS015	Doni Kurniawan	1996-09-16	L	Jl. Baskoro No. 5, Ambarawa	081311110015	doni.kurniawan@gmail.com	AB	2024-02-12 09:45:00

QUERY JOIN DAN AGREGAT

```
-- [10] AGREGAT + GROUP BY + HAVING:
-- Dokter dengan total pasien > 1 beserta total pendapatan

SELECT
    d.id_dokter,
    d.nama_dokter,
    d.spesialisasi,
    COUNT(DISTINCT pd.id_pasien) AS total_pasien,
    COUNT(pd.id_daftar)          AS total_kunjungan,
    SUM(tg.biaya_dokter)         AS total_pendapatan,
    AVG(tg.total_tagihan)        AS rata_rata_tagihan
FROM dokter d
JOIN pendaftaran pd ON d.id_dokter = pd.id_dokter
JOIN rekam_medis rm ON pd.id_daftar = rm.id_daftar
JOIN tagihan tg ON rm.id_rekam = tg.id_rekam
GROUP BY d.id_dokter, d.nama_dokter, d.spesialisasi
HAVING COUNT(DISTINCT pd.id_pasien) > 1
ORDER BY total_pendapatan DESC;
```

SELECT, COUNT, SUM, AVG : Fungsi agregat untuk menghitung total & rata-rata

FROM dokter d : Tabel utama: data dokter

JOIN pendaftaran : Menggabungkan data kunjungan pasien

JOIN rekam_medis : Menggabungkan data pemeriksaan

JOIN tagihan : Menggabungkan data pembayaran

GROUP BY : Mengelompokkan hasil per dokter

HAVING COUNT > 1 : Hanya tampilkan dokter dengan lebih dari 1 pasien

ORDER BY total_pendapatan : Hanya tampilkan dokter dengan lebih dari 1 pasien

Query ini menggabungkan 3 tabel menggunakan JOIN dan menggunakan fungsi agregat COUNT, SUM, AVG untuk menghasilkan rekap pendapatan per dokter.

OUTPUT HASIL QUERY JOIN DAN AGREGAT

	id_dokter	nama_dokter	spesialisasi	total_pasien	total_kunjungan	total_pendapatan	rata_rata_tagihan
▶	D001	dr. Hendra Kusuma, Sp.PD	Penyakit Dalam	5	5	2050000.00	797000.000000
	D004	dr. Dewi Permatasari, Sp.JP	Jantung & Pembuluh Darah	3	3	2000000.00	861666.666667
	D003	dr. Bambang Wibowo, Sp.OG	Obstetri & Ginekologi	2	2	1000000.00	1074000.000000
	D002	dr. Siti Rahayu, Sp.A	Anak	2	2	300000.00	272125.000000

VIEW & STORED PROCEDURE

VIEW 1 (v_ringkasan_pasien)

```
5 -- VIEW 1: Ringkasan data pasien + kunjungan terakhir
6 • CREATE OR REPLACE VIEW v_ringkasan_pasien AS
7 SELECT
8     ps.id_pasien,
9     ps.nama_lengkap,
10    TIMESTAMPDIFF(YEAR, ps.tanggal_lahir, CURDATE()) AS usia,
11    ps.jenis_kelamin,
12    ps.golongan_darah,
13    ps.no_telepon,
14    COALESCE(kj.total_kunjungan, 0) AS total_kunjungan,
15    kj.kunjungan_terakhir,
16    COALESCE(tg.total_tagihan_keseluruhan, 0) AS total_tagihan_keseluruhan
17 FROM pasien ps
18 LEFT JOIN (
19     SELECT id_pasien,
20            COUNT(*) AS total_kunjungan,
21            MAX(tanggal_daftar) AS kunjungan_terakhir
22     FROM pendaftaran
23     GROUP BY id_pasien
24 ) kj ON ps.id_pasien = kj.id_pasien
25 LEFT JOIN (
```

CREATE OR REPLACE

VIEW

Membuat atau memperbarui virtual table v_ringkasan_pasien tanpa menyimpan data fisik

TIMESTAMPDIFF(YEAR, ...)

Menghitung usia pasien secara otomatis dari tanggal lahir hingga hari ini

COALESCE(..., 0)

Jika pasien belum punya kunjungan/tagihan, nilai tampil sebagai 0 (bukan NULL)

LEFT JOIN subquery

Menggabungkan data kunjungan & tagihan dari subquery terpisah ke data pasien utama

CASE WHEN ... END AS status_stok

Memberi label kondisi stok: Habis, Hampir Habis, Cukup, atau Aman berdasarkan perbandingan dengan stok_minimum

stok * harga → nilai_stok

Menghitung nilai total stok obat (jumlah × harga satuan) secara langsung di dalam view

ORDER BY FIELD(...)

Mengurutkan hasil berdasarkan urutan prioritas kustom:
Habis → Hampir Habis → Cukup → Aman

Tanpa WHERE / GROUP BY

View ini memuat semua obat sekaligus filter & agregasi dilakukan saat view dipanggil

VIEW 2—v_stok_obat

```
34 -- VIEW 2: Monitoring stok obat dan status ketersediaan
35 • CREATE OR REPLACE VIEW v_stok_obat AS
36 SELECT
37     id_obat,
38     nama_obat,
39     kategori,
40     satuan,
41     harga,
42     stok,
43     stok_minimum,
44     CASE
45         WHEN stok = 0 THEN 'Habis'
46         WHEN stok <= stok_minimum THEN 'Hampir Habis'
47         WHEN stok <= stok_minimum * 2 THEN 'Cukup'
48         ELSE 'Aman'
49     END AS status_stok,
50     (stok * harga) AS nilai_stok
51 FROM obat
52 ORDER BY
53     FIELD(
54         CASE
55             WHEN stok = 0 THEN 'Habis'
```


VIEW 1: Ringkasan data pasien + kunjungan terakhir

	id_pasien	nama_lengkap	usia	jenis_kelamin	golongan_darah	no_telepon	total_kunjungan	kunjungan_terakhir	total_tagih
▶	PS001	Budi Santoso	48	L	A	081311110001	3	2026-06-22	321000.00
	PS002	Siti Aminah	40	P	O	081311110002	2	2024-04-15	236250.00
	PS003	Ahmad Fauzi	33	L	B	081311110003	2	2024-04-15	382000.00
	PS004	Dwi Lestari	36	P	AB	081311110004	2	2024-04-16	436000.00
	PS005	Heru Prasetyo	50	L	A	081311110005	2	2024-04-16	471000.00
	PS006	Nur Hidayah	30	P	O	081311110006	1	2024-04-03	194800.00
	PS007	Wahyu Triadi	38	L	B	081311110007	1	2024-04-04	9075000.00
	PS008	Eka Putri Rahayu	24	P	A	081311110008	2	2024-04-15	308000.00
	PS009	Joko Susilo	60	L	O	081311110009	2	2024-04-15	825000.00
	PS010	Ani Suryani	46	P	AB	081311110010	2	2024-04-16	614000.00
	PS011	Rizki Maulana	26	L	A	081311110011	1	2024-04-08	0.00
	PS012	Fitriani Dewi	32	P	B	081311110012	2	2024-04-16	1712000.00
	PS013	Supardi	68	L	O	081311110013	1	2024-04-09	857000.00

VIEW 2: Monitoring stok obat dan status ketersediaan

	id_obat	nama_obat	kategori	satuan	harga	stok	stok_minimum	status_stok	nilai_stok
▶	OB001	Amoxicillin 500mg	Antibiotik	Kapsul	3500.00	5	50	Hampir Habis	17500.00
	OB004	Amlodipine 5mg	Antihipertensi	Tablet	3800.00	30	60	Hampir Habis	114000.00
	OB009	NaCl 0.9% 500ml	Cairan Infus	Kolf	25000.00	50	30	Cukup	1250000.00
	OB019	Salbutamol 2mg	Bronkodilator	Tablet	2300.00	60	45	Cukup	138000.00
	OB002	Paracetamol 500mg	Analgesik	Tablet	850.00	1200	100	Aman	1020000.00
	OB003	Metformin 500mg	Antidiabetes	Tablet	2200.00	800	80	Aman	1760000.00
	OB005	Omeprazole 20mg	Antasida	Kapsul	4500.00	450	50	Aman	2025000.00
	OB006	Simvastatin 20mg	Statin	Tablet	5200.00	350	40	Aman	1820000.00
	OB007	Ceftriaxone 1gr	Antibiotik	Vial	45000.00	150	20	Aman	6750000.00
	OB008	Ringer Laktat	Cairan Infus	Kolf	28000.00	200	30	Aman	5600000.00
	OB010	Ranitidin 150mg	Antasida	Tablet	1800.00	700	70	Aman	1260000.00
	OB011	Dexamethasone 0....	Kortikosteroid	Tablet	2100.00	400	40	Aman	840000.00
	OB012	Captopril 25mg	Antihipertensi	Tablet	2500.00	550	55	Aman	1375000.00
	OB013	Cetirizine 10mg	Antihistamin	Tablet	3200.00	600	60	Aman	1920000.00
	OB014	Ibuprofen 400mg	NSAID	Tablet	2800.00	800	80	Aman	2240000.00
	OB015	Insulin Regular	Antidiabetes	Vial	85000.00	80	15	Aman	6800000.00

STORED PROCEDURE

Prosedur ini digunakan untuk mendaftarkan pasien ke poli tertentu sekaligus membuat nomor antrian secara otomatis.

Parameter Input : Menerima data pendaftaran seperti ID daftar, ID pasien, ID poli, ID dokter, tanggal, dan jam pendaftaran.

Start Transaction : Memulai transaksi untuk memastikan seluruh proses pendaftaran berjalan sebagai satu kesatuan.

Hitung no antrian otomatis : Mencari nomor antrian terakhir pada poli, dokter, dan tanggal yang sama, kemudian menambahkan 1.

INSERT INTO pendaftaran : Menyimpan data pendaftaran pasien ke tabel pendaftaran dengan status "menunggu".

COMMIT: Menyimpan seluruh perubahan ke database jika tidak terjadi kesalahan.

```
-- Prosedur: Proses pendaftaran pasien baru sekaligus
-- membuat tagihan awal (biaya dokter)
DELIMITER $$

CREATE PROCEDURE sp_daftar_pasien (
    IN p_id_daftar VARCHAR(10),
    IN p_id_pasien VARCHAR(10),
    IN p_id_poli VARCHAR(10),
    IN p_id_dokter VARCHAR(10),
    IN p_tgl_daftar DATE,
    IN p_jam_daftar TIME,
    OUT p_no_antrian INT,
    OUT p_pesan VARCHAR(255)
)
BEGIN
    DECLARE v_antrian INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_pesan = 'ERROR: Gagal mendaftar, terjadi kesalahan sistem.';
    END;

    START TRANSACTION;

    -- Hitung nomor antrian otomatis
    SELECT IFNULL(MAX(no_antrian), 0) + 1
    INTO v_antrian
    FROM pendaftaran
    WHERE id_poli = p_id_poli
    AND id_dokter = p_id_dokter
    AND tanggal_daftar = p_tgl_daftar;

    -- Insert ke tabel pendaftaran
    INSERT INTO pendaftaran
    (id_daftar, id_pasien, id_poli, id_dokter,
    tanggal_daftar, jam_daftar, no_antrian, status)
    VALUES
    (p_id_daftar, p_id_pasien, p_id_poli, p_id_dokter,
    p_tgl_daftar, p_jam_daftar, v_antrian, 'menunggu');

    COMMIT;

    SET p_no_antrian = v_antrian;
    SET p_pesan = CONCAT('Pendaftaran berhasil. No. Antrian: ', v_antrian);
END$$

DELIMITER ;
```

```
-- Contoh pemanggilan stored procedure:
CALL sp_daftar_pasien(
    'REG050',
    'PS001',
    'P005',
    'D004',
    CURDATE(),
    CURTIME(),
    @antrian,
    @pesan
);

SELECT * FROM pendaftaran;

SELECT *
FROM pendaftaran
WHERE id_daftar = 'REG021';
```



[illegible]

	id_daftar	id_pasien	id_poli	id_dokter	tanggal_daftar	jam_daftar	no_antrian	status
▶	REG001	PS001	P001	D001	2024-04-01	07:45:00	1	selesai
	REG002	PS002	P002	D002	2024-04-01	08:00:00	1	selesai
	REG003	PS003	P001	D001	2024-04-02	08:10:00	1	selesai
	REG004	PS004	P003	D003	2024-04-02	09:00:00	1	selesai
	REG005	PS005	P004	D004	2024-04-03	08:30:00	1	selesai
	REG006	PS006	P008	D008	2024-04-03	09:15:00	1	selesai
	REG007	PS007	P005	D005	2024-04-04	07:30:00	1	selesai
	REG008	PS008	P002	D002	2024-04-04	10:00:00	1	selesai
	REG009	PS009	P001	D001	2024-04-05	08:45:00	1	selesai
	REG010	PS010	P004	D004	2024-04-05	10:30:00	1	selesai
	REG011	PS011	P006	D006	2024-04-08	09:00:00	1	selesai
	REG012	PS012	P003	D003	2024-04-08	09:30:00	1	selesai
	REG013	PS013	P001	D001	2024-04-09	08:00:00	1	selesai
	REG014	PS014	P008	D008	2024-04-09	10:00:00	1	selesai
	REG015	PS015	P007	D007	2024-04-10	08:30:00	1	selesai
	REG016	PS016	P001	D001	2024-04-10	09:00:00	1	selesai
	REG017	PS017	P004	D004	2024-04-11	09:00:00	1	selesai

[illegible]

TRIGGER

Trigger ini digunakan untuk mengurangi stok obat secara otomatis setelah data resep ditambahkan serta mencegah transaksi ketika stok tidak mencukupi.

```
-- Trigger: Kurangi stok obat otomatis setiap kali
-- resep baru ditambahkan; batalkan jika stok tidak cukup
DELIMITER $$

CREATE TRIGGER trg_kurangi_stok_obat
AFTER INSERT ON resep
FOR EACH ROW
BEGIN
    DECLARE v_stok_sekarang INT;

    SELECT stok INTO v_stok_sekarang
    FROM obat
    WHERE id_obat = NEW.id_obat;

    IF v_stok_sekarang < NEW.jumlah THEN
        SIGNAL SQLSTATE '45000'
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stok obat tidak mencukupi untuk resep ini.';
    ELSE
        UPDATE obat
        SET stok = stok - NEW.jumlah
        WHERE id_obat = NEW.id_obat;
    END IF;
END$$

DELIMITER ;

-- Verifikasi trigger (cek stok OB001 sebelum & sesudah insert resep baru):
SELECT id_obat, nama_obat, stok FROM obat WHERE id_obat = 'OB001';

INSERT INTO resep VALUES('RS023', 'RM001', 'OB001', 20, '2x sehari sesudah makan');
SELECT id_obat, nama_obat, stok FROM obat WHERE id_obat = 'OB001';
```

AFTER INSERT ON resep : Aktif setelah data resep baru ditambahkan

FOR EACH ROW : Trigger dijalankan untuk setiap baris data resep yang diinput.

SELECT stok INTO v_stok_sekarang : Mengambil nilai stok obat saat ini dari tabel obat dan menyimpannya ke variabel sementara.

IF stok < jumlah : Mengecek apakah stok yang tersedia mencukupi jumlah obat yang diresepkan.

SIGNAL SQLSTATE : Menampilkan pesan error jika stok tidak cukup.

UPDATE stok Mengurangi stok obat secara otomatis sesuai jumlah yang diresepkan.

	id_obat	nama_obat	stok
▶	OB001	Amoxicillin 500mg	25

	id_obat	nama_obat	stok
▶	OB001	Amoxicillin 500mg	5
*	NULL	NULL	NULL

FUNCTION

Menyimpan logika perhitungan yang dapat dipanggil kembali dan mengembalikan satu nilai secara otomatis

```
DELIMITER $$  
CREATE FUNCTION fn_usia_pasien (  
    p_id_pasien VARCHAR(10)  
)  
RETURNS INT  
DETERMINISTIC  
READS SQL DATA  
BEGIN  
    DECLARE v_tgl_lahir DATE;  
    DECLARE v_usia INT;  
  
    SELECT tanggal_lahir  
    INTO v_tgl_lahir  
    FROM pasien  
    WHERE id_pasien = p_id_pasien;  
  
    SET v_usia = TIMESTAMPDIFF(YEAR, v_tgl_lahir, CURDATE());  
  
    RETURN v_usia;  
END$$  
DELIMITER ;
```

DELIMITER

Mengubah tanda pemisah sementara agar MySQL tidak bingung membaca titik koma di dalam body function

RETURNS INT + DETERMINISTIC

Menyatakan function ini selalu mengembalikan bilangan bulat (usia), dan hasilnya konsisten untuk input yang sama

READS SQL DATA

Memberitahu MySQL bahwa function ini hanya membaca data (tidak mengubah tabel), penting untuk keamanan & optimasi

DECLARE + SELECT INTO

Mendeklarasikan variabel lokal, lalu mengambil tanggal lahir dari tabel pasien ke dalam variabel tersebut

FUNCTION

```
-- Contoh pemanggilan function:
SELECT
  id_pasien,
  nama_lengkap,
  tanggal_lahir,
  fn_usia_pasien(id_pasien) AS usia,
  CASE
    WHEN fn_usia_pasien(id_pasien) < 12 THEN 'Anak'
    WHEN fn_usia_pasien(id_pasien) BETWEEN 12 AND 17 THEN 'Remaja'
    WHEN fn_usia_pasien(id_pasien) BETWEEN 18 AND 59 THEN 'Dewasa'
    ELSE 'Lansia'
  END AS kategori_usia
FROM pasien
ORDER BY usia DESC;
```

OUTPUT FUNCTION

	id_pasien	nama_lengkap	tanggal_lahir	usia	kategori_usia
▶	PS028	Sulastri	1955-03-14	71	Lansia
	PS013	Supardi	1958-06-04	68	Lansia
	PS021	Sunarto	1962-05-17	64	Lansia
	PS009	Joko Susilo	1965-08-19	60	Lansia
	PS024	Lastri Handayani	1969-02-08	57	Dewasa
	PS017	Teguh Wibowo	1971-07-14	54	Dewasa
	PS005	Heru Prasetyo	1975-09-08	50	Dewasa
	PS001	Budi Santoso	1978-04-12	48	Dewasa
	PS020	Nining Rahayu	1977-11-30	48	Dewasa
	PS027	Tanto Priyambodo	1979-06-21	47	Dewasa
	PS010	Ani Suryani	1980-05-07	46	Dewasa
	PS016	Halimah Tussadivah	1983-12-01	42	Dewasa

fn_usia_pasien(id_pasien) AS usia

Memanggil function yang sudah dibuat untuk setiap baris
— hasilnya langsung dijadikan kolom baru bernama usia

CASE WHEN ... END AS kategori_usia

Menggunakan hasil function sebagai dasar
pengelompokan usia: Anak (<12), Remaja (12–17), Dewasa
(18–59), Lansia (60+)

ORDER BY usia DESC

Mengurutkan hasil dari pasien tertua ke termuda
menggunakan kolom alias hasil pemanggilan function

KESIMPULAN

- Berhasil merancang basis data relasionalSIMRS menggunakan MySQL.
- Terdiri dari 11 entitas utama yang terhubung via PK & FK.
- Proses normalisasi hingga 3NF memastikan data bebas dari redundansi & anomali.
- Query, VIEW, Stored Procedure, Trigger, dan Function berhasil diimplementasikan.
- Sistem mampu mengelola seluruh alur: pendaftaran → rekam medis → resep → tagihan.

SARAN

- Menambahkan fitur manajemen pengguna & hak akses
- Integrasi jadwal dokter secara otomatis
- Mengembangkan modul laboratorium & apotek supaya lebih lengkap
- Mengembangkan menjadi aplikasi web atau mobile

A collection of medical supplies including a multi-day pill organizer, a stethoscope, blue gloves, a pulse oximeter, a blood pressure cuff, a digital thermometer, and a syringe, all arranged on a light-colored surface.

TERMAKASIH